

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



Principles of Database Management – IT079IU
PROJECT PROGRESS REPORT (D2)

Topic 9: Vehicle Registration Authority

Group 07 Member List

1. Võ Tuấn Kiệt	ITCSIU25029	Team Leader
2. Trần Nguyễn Thành An	ITCSIU25001	Query Developer
3. Phan Hoàng An	ITCSIU25002	Tester and Writer
4. Bùi Gia Bảo	ITCSIU25007	Java Developer
5. Trần Quốc Bảo	ITCSIU25008	Normalization Leader
6. Trần Khánh Duy	ITCSIU25014	Java Developer
7. Nguyễn Quang Khải	ITCSIU25019	Database Developer
8. Phan Trần Nguyên Khang	ITCSIU25021	Database Developer
9. Võ Đăng Khoa	ITCSIU25022	Query Developer
10. Võ Nguyễn Quang Minh	ITCSIU25034	Normalization Leader
11. Võ Nhật Nam	ITCSIU25038	Data Modeler
12. Nguyễn Khả Toàn Tấn	ITCSIU25046	Java Developer
13. Phùng Anh Tuấn	ITCSIU25050	Data Modeler

Instructor: Assoc. Prof. Dr. Nguyen Thi Thuy Loan

Table of Contents

Part 1: Updated Information	3
1.1. Refinement of Work Allocation & Schedule	3
1.2. Technological Stack Consolidation	3
1.3. Domain Scope Enhancement	3
Part 2: Database Design	4
2.1. Detailed Business Rules	4
2.2. Conceptual Design (ER Diagram)	5
2.3. Logical Database Design (Relational Schema)	6
Lecture 3 Mapping Rules Applied:	7
<i>GitHub link for checking our group process</i>	7

Part 1: Updated Information

With the submission of the Project Proposal Report (D1), our group has refined and finalized several project aspects based on initial architectural assessments and detailed domain analysis:

1.1. Refinement of Work Allocation & Schedule

- Our team consolidated task distributions across all 13 members to strictly align with the university's submission milestones (Deliverable D2 at Week 5, and Final Submission D3, D5, D6 at Week 10).
- Specific sub-teams have commenced concurrent workflows: the Modeling team finalized the ERD and Relational Schema, the Database Engineers prepared the initial DDL scripts, and the Java team completed application UI mockups and JDBC connection modules.

1.2. Technological Stack Consolidation

- DBMS: Standardized on MySQL 8.0+ using the InnoDB storage engine to guarantee full ACID transactional compliance and foreign key referential actions (ON DELETE CASCADE, ON UPDATE CASCADE).
- Application Architecture: Java 21 LTS or Java 25 with pure JDBC (mysql-connector-j-8.3.0) and the Data Access Object (DAO) pattern to enforce complete separation between business logic and SQL execution.
- Modeling Tools: Conceptual ERD designed using ERDPlus and validated through relational mapping guidelines from Week 3's Lecture.

1.3. Domain Scope Enhancement

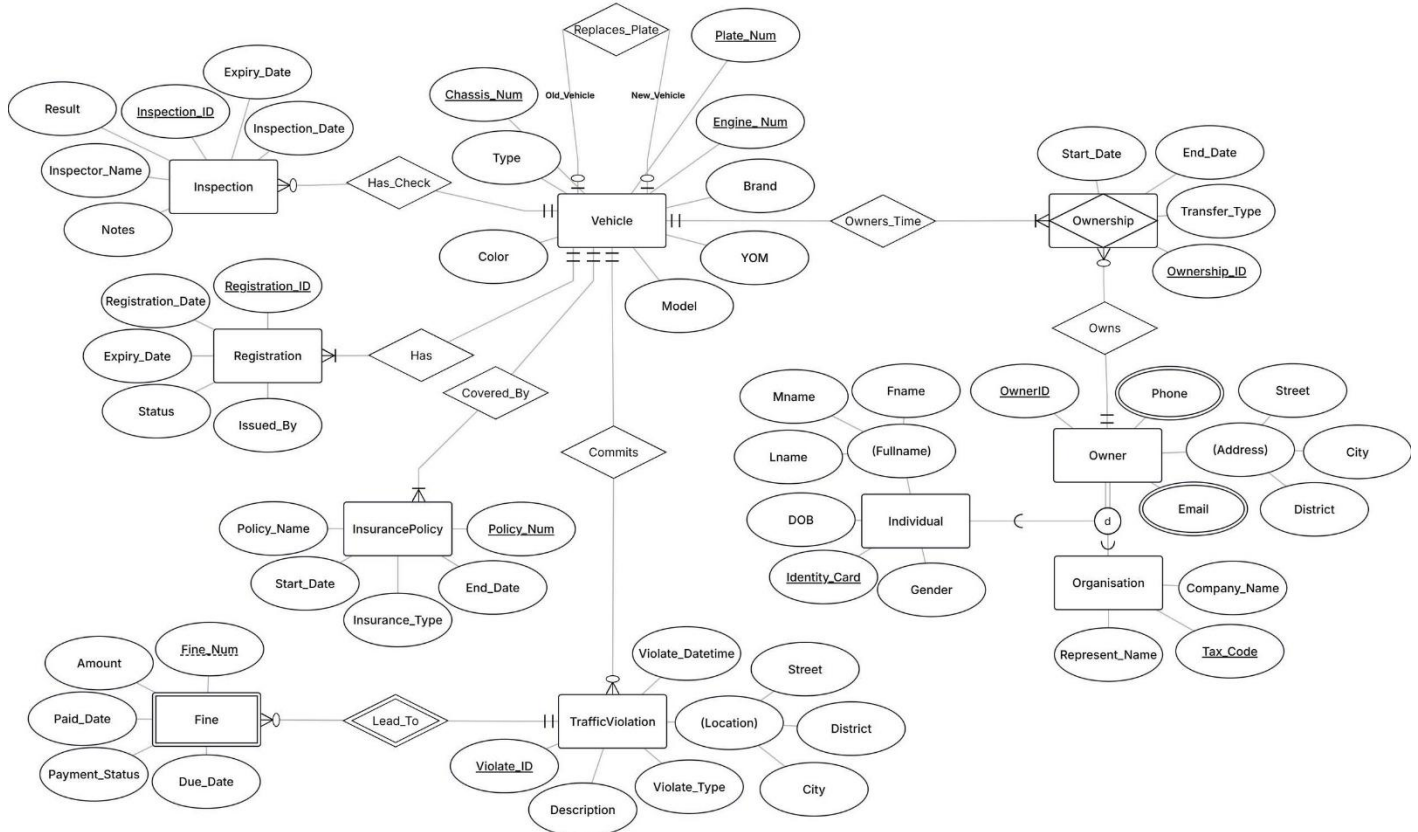
- Refined vehicle transfer mechanics by explicitly treating Ownership as an associative entity capturing temporal changes (Start_Date, End_Date, Transfer_Type).
- Implemented a partial 1-to-1 recursive relationship Replaces_Plate on Vehicle to allow vehicle owners to transfer retained license plates to newly acquired vehicles.
- Modeled Fine as a weak entity strictly dependent on TrafficViolation, enforcing that fine notices cannot exist independently of a recorded infraction.

Part 2: Database Design

2.1. Detailed Business Rules

- **BR1: Vehicle Identification:** Each vehicle is uniquely identified by its license plate number. In addition, each vehicle possesses a chassis number and an engine number, both of which must be unique across all vehicles.
- **BR2: Owner Hierarchy:** An owner must be categorized as either an individual or an organization. This specialization is total and disjoint, meaning an owner cannot belong to any other category nor be both simultaneously. An individual is identified by a national ID, whereas an organization is identified by a tax code.
- **BR3: Ownership Tracking:** Over time, a vehicle may undergo multiple ownership changes, and an owner may own multiple vehicles. This historical relationship is maintained via the ownership associative entity, which records the start date, end date, and transfer type for each ownership period.
- **BR4: Registration Certificates:** A vehicle may be issued multiple registration certificates over its lifespan. However, each registration certificate belongs to exactly one vehicle.
- **BR5: Insurance Policies:** A vehicle must be covered by at least one active insurance policy to legally operate. Each insurance policy covers one and only one vehicle.
- **BR6: Technical Inspection:** A vehicle may undergo zero or more technical inspections. Each inspection record pertains to exactly one vehicle.
- **BR7: Traffic Violations:** A vehicle can be involved in zero or more traffic violations. Each traffic violation records the specific timestamp, location, violation type, and description, and is associated with exactly one offending vehicle.
- **BR8: Fines and Weak Entity Dependency:** A traffic violation may result in zero or more fine notices. A fine notice is a weak entity whose existence is entirely dependent on its corresponding traffic violation.
- **BR9: License Plate Replacement- Recursive Relationship:** Upon selling or transferring a vehicle, the owner may retain the existing license plate number to assign it to a newly acquired vehicle. This represents a 1-to-1 partial recursive relationship on the vehicle entity between a new vehicle and a previous vehicle.
- **BR10: Contact Information:** An owner may register multiple contact phone numbers and multiple email addresses.

2.2. Conceptual Design (ER Diagram)



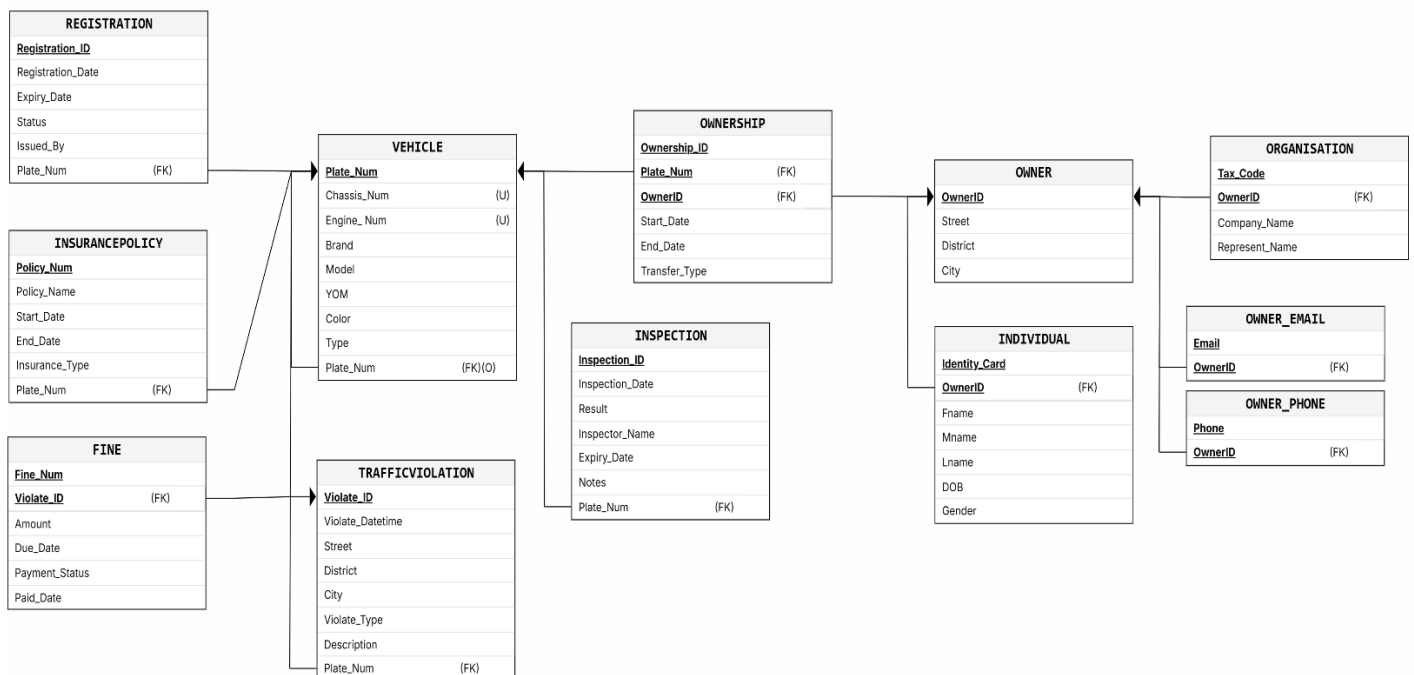
ER Diagram for Vehicle Registration Authority by Anh Tuan and Nhat Nam

Table of Relationships:

Relationship	Entities Joined	Ratio	Participation	Justification from Business Rules
IS_A	Owner – Individual/Organisation	1 – 1	Mandatory, Disjoint	BR2: Every owner must be either an individual or an organisation, never both.
Owners_Time	Vehicle – Ownership	1 – M	Mandatory: Vehicle Mandatory: Ownership	BR3: A vehicle has historical ownership periods; an ownership record must belong to a vehicle.
Owns	Owner – Ownership	1 – M	Mandatory: Ownership Optional: Owner	BR3: An owner can own multiple vehicles over time; an ownership period must belong to an owner.
Has	Vehicle – Registration	1 – M	Mandatory: Vehicle Mandatory: Registration	BR4: Each registration certificate belongs to exactly one vehicle; vehicles have multiple renewals.
Covered_By	Vehicle – InsurancePolicy	1 – M	Mandatory: Vehicle Mandatory: InsurancePolicy	BR5: A vehicle must have insurance to operate; each policy covers exactly one vehicle.
Has_Check	Vehicle – Inspection	1 – M	Mandatory: Inspection Optional: Vehicle	BR6: New vehicles or motorbikes may not have inspections yet; an

				inspection belongs to one vehicle.
Commits	Vehicle – TrafficViolation	1 – M	Mandatory: TrafficViolation Optional: Vehicle	BR7: Law-abiding vehicles have zero violations; each violation is committed by one vehicle.
Leads_To	TrafficViolation – Fine	1 – M	Mandatory: Fine Optional: TrafficViolation	BR8: Some violations carry warnings without fines; a fine cannot exist without a violation.
Replaces_Plate	Vehicle (Old_Vehicle) – Vehicle (New_Vehicle)	1 – 1	Optional: Vehicle Optional: Vehicle	BR9: Plate retention occurs optionally when an owner reassigns an old plate to a new vehicle.

2.3. Logical Database Design (Relational Schema)



Relational Schema Diagram by Anh Tuan and Nhat Nam

a) Strong entities

- Vehicle: **Plate_Num** | Chassis_Num | Engine_Num | Brand | Model | YOM | Color | Type | Plate_Num (FK) (O).
- Inspection: **Inspection_ID** | Inspection_Date | Result | Inspector_Name | Expiry_Date | Notes | Plate_Num (FK).
- Ownership: **Ownership_ID** | Start_Date | End_Date | Transfer_Type | Plate_Num (FK) | OwnerID (FK).
- Registration: **Registration_ID** | Registration_Date | Status | Expiry_Date | Issued_By | Plate_Num (FK).
- InsurancePolicy: **Policy_Num** | Policy_Name | Start_Date | End_Date | Insurance_Type | Plate_Num (FK).
- TrafficViolation: **Violate_ID** | Violate_Datetime | Street | District | City | Violate_Type | Description | Plate_Num (FK).

b) Weak entity

Fine: **Fine_Num** (Partial Key – PK) | Paid_Date | Amount | Due_Date | Payment_Status | **Violate_ID** (PK) (FK).

Primary Key of Fine is a composite key {Violate_ID, Fine_Num}.

c) Multivalued Attributes:

- Owner_Phone: Phone (PK) | OwnerID (PK) (FK).
- Owner_Email: Email (PK) | OwnerID (PK) (FK).

d) Specialization:

- Owner: OwnerID | Street | City | District.
- Individual: Identity_Card | OwnerID (PK) (FK) | Fname | Mname | Lname | DOB | Gender.
- Organisation: Tax_Code | OwnerID (PK) (FK) | Company_Name | Represent_Name.

Lecture 3 Mapping Rules Applied:

- **Rule 1 (Regular Entities):** Mapped Vehicle, Owner, Registration, InsurancePolicy, Inspection, TrafficViolation into individual tables with their respective primary keys.
- **Rule 2 (Weak Entities):** Mapped Fine by including its discriminator Fine_Num combined with the primary key of the owner entity TrafficViolation (Violate_ID) to form the composite primary key (Violate_ID, Fine_Num).
- **Rule 3 (1:N Binary Relationships):** Mapped Has, Covered_By, Has_Check, and Commits by embedding Plate_Num as a foreign key on the "Many" side entities.
- **Rule 4 (M:N Binary Relationships / Associative Entities):** Mapped the *M:N* relationship between Vehicle and Owner through the Ownership table containing foreign keys Plate_Num and OwnerID, along with relationship attributes (Start_Date, End_Date, Transfer_Type).
- **Rule 5 (Multivalued Attributes):** Decomposed Phone and Email of Owner into separate relations Owner_Phone and Owner_Email, keyed by (OwnerID, Phone) and (OwnerID, Email).
- **Rule 6 (Specialization / ISA Hierarchies):** Mapped the total disjoint ISA hierarchy into separate tables Individual and Organisation, sharing the primary key OwnerID as a foreign key referencing Owner.
- **Rule 7 (1:1 Recursive Relationships):** Mapped Replaces_Plate by introducing a nullable foreign key Prev_Plate_Num within Vehicle self-referencing Plate_Num.

GitHub link for checking our group process

[frozenhero1064/PDM.VehicleRegistrationG7: G7 - PDM Project Sem 1 26_27 Ms. Loan](https://github.com/frozenhero1064/PDM.VehicleRegistrationG7:G7-PDM-Project-Sem12627MsLoan)

Or you can scan the QR code to access:

